University of Toronto at Scarborough
**CSCC37—Numerical Algorithms for Computational Mathematics, Fall 2024**

# Assignment #1: FP Systems / Vector Norms

*Due: October 12, 2024 at 11:45 p.m.*
*This assignment is worth 10% of your final grade.*

**Warning:** Your electronic submission on *MarkUs* affirms that this assignment is your own work and no one else's, and is in accordance with the University of Toronto Code of Behaviour on Academic Matters, the Code of Student Conduct, and the guidelines for avoiding plagiarism in CSCC37.

This assignment is due by 11:45 p.m. October 12. If you haven't finished by then, you may hand in your assignment late with a penalty as specified in the course information sheet.

[10] 1. Machine precision $\epsilon$ is defined as the smallest positive non-normalized floating point number such that $1 + \epsilon > 1$. Prove that $\epsilon$ is the bound for relative round-off. In other words, prove that:

$$\epsilon = \begin{cases} b^{1-t} & \text{chopping} \\ \frac{1}{2}b^{1-t} & \text{rounding} \end{cases}$$

where $b$ is the base of the computer's floating point number system and $t$ is the mantissa length.

[10] 2. Let $x, y \in \mathbb{R}$. Recall that $fl(x), fl(y) \in \mathbb{R}_b(t, s)$ denote the floating-point representations of $x$ and $y$, respectively, where $fl(x) = x(1 - \delta_x)$, $fl(y) = y(1 - \delta_y)$, and $\delta_x$, $\delta_y$ quantify the relative roundoff errors in the respective representations.

In lecture we showed that a typical computer estimates the product of $x$ and $y$ as

$$fl(fl(x) \cdot fl(y)) = (x \cdot y)(1 - \delta_\cdot)$$

where $|\delta_\cdot| \leq 3$ eps. Derive a similar error bound for computer division. **Clearly state any assumptions made when simplifying expressions during the analysis.**

[15] 3. Show that $fl(x^k) = x^k(1 + \delta)^{k-1}$, where $|\delta| \leq \epsilon$, if $x$ is a floating-point machine number in a computer that has unit roundoff $\epsilon$.

**Hints:** There is no initial roundoff error associated with storing $x$, as $x$ is already a floating point number. All roundoff arises from the multiplications used in computing $x^k$. The "unit roundoff $\epsilon$" is the bound for roundoff assuming chopping. More precisely, $x \in \mathbb{R}_b(t, s)$ and $\epsilon = b^{1-t}$. The roundoff errors occurring as the exponentiation proceeds are not the same. For example, the error $\delta_1$ arising from $x \cdot x$ differs from the error $\delta_2$ arising from $(x \cdot x) \cdot x$. **Your analysis must take into account that the roundoff errors differ at each step of the exponentiation.**

[20] 4. Consider the quadratic equation

$$ax^2 + bx + c = 0,$$

where $a, b$, and $c \in \mathbb{R}$. (Assume $a \neq 0$, although this is an exception you would need to consider.)

As you know, the quadratic equation has two roots

$$r_{\pm} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}.$$

One could use this formula in a very simple program to solve the quadratic. The only slight complication you would need to handle arises from the observation that the *discriminant* $b^2 - 4ac$ may be negative, in which case the two roots are a complex conjugate pair with real and imaginary parts $-b/2a$ and $\sqrt{4ac - b^2}/2a$, respectively. All that is required to handle this in a simple program is a selection statement to separate the case of two real roots from that of a complex conjugate pair.

However, there are other serious computational problems that must be addressed as well if you are to develop a robust program. For instance, there are at least two operations in the above formula that could result in *subtractive cancellation*. Also, the computation of the discriminant $b^2 - 4ac$ may result in an intermediate *overflow* or *underflow*, even though the roots do not.

Identify the ranges of $a, b$ and $c$ where these exceptions occur (a general range will suffice, you do not have details of a specific floating point system), and in each case provide an alternate expression to compute the roots which avoids the problem.

[10] 5. Each of the following expressions defines a mapping from $x \in \mathbb{R}^n$ to $||x|| \in \mathbb{R}$. In each case, prove or disprove that the mapping is a vector norm.

(a) $||x|| = \max\{|x_1|, |x_2|, |x_3|, \ldots, |x_n|\}$

(b) $||x|| = \max\{|x_2|, |x_3|, \ldots, |x_n|\}$

(c) $||x|| = n$

(d) $||x|| = \sum_{i=1}^{n} |x_i|^3$

(e) $||x|| = \left[\sum_{i=1}^{n} |x_i|^{\frac{1}{2}}\right]^2$

[total: 65 marks]